

TestDoc Agent

Gerador de documentação de testes orientado por requisitos

Bárbara Mota

Eduardo Carlos Barboza Neto

Raphael Torres Cavalcanti Ferreira

Tópicos

1. Contexto
2. O Problema x Solução
3. Agents
4. Tools e patterns
5. Avaliação da Solução
6. Conclusão
7. Referências

Contexto

Contexto

- Requisitos precisam ser transformados em cenários de teste claros e verificáveis
- A criação de casos de teste ainda depende de esforço manual e é propensa a inconsistências
- A ausência de estrutura dificulta padronizar cenários, cobertura e tipos de validação
- Há demanda por acelerar a especificação de testes sem comprometer qualidade e cobertura

O Problema x Solução

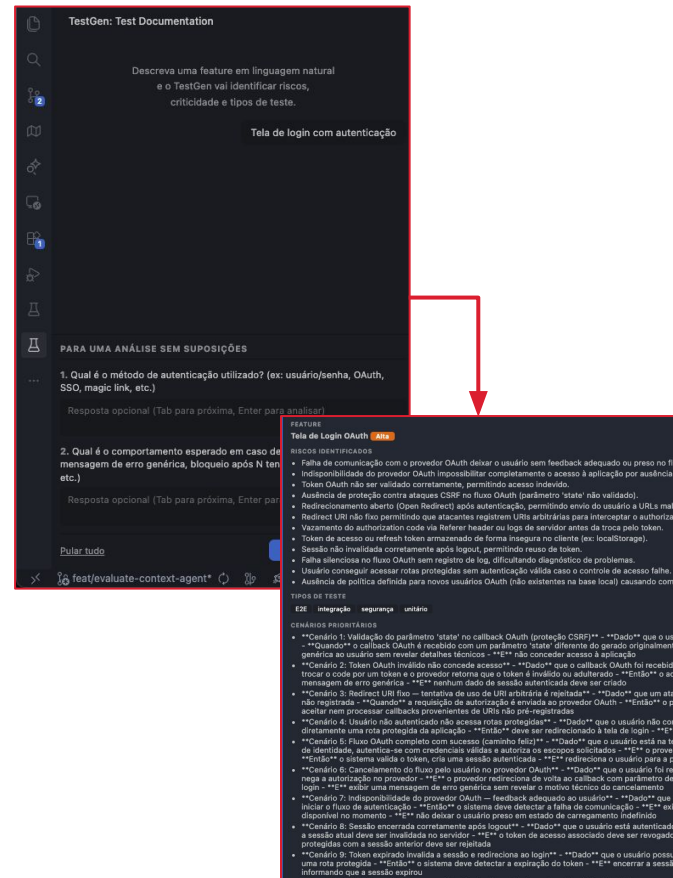
O Problema

- **Input humano variável:** cada dev tende a descrever features de forma diferente
- **Análise de risco manual:** depende do conhecimento tácito do QA
- **Inconsistência:** projetos diferentes, resultados diferentes para features similares
- **Tempo:** planejar testes de uma feature complexa leva horas
- **Rastreabilidade:** difícil ligar risco → tipo de teste → cenário de forma auditável

A solução

Gerador inteligente de documentação de testes para features de software:

- Recebe descrições funcionais em texto livre ou JSON estruturado
- Atua no apoio à análise de QA antes da execução dos testes
- Usa agentes especializados para interpretar, analisar riscos, propor testes e revisar a estratégia
- Entrega um plano com riscos, criticidade, tipos de teste e cenários priorizados



Agents

Solução — Pipeline Multi-Agente



Tools e patterns

Padrões de Agentes de IA Utilizados

Backend

**Python com FastAPI
+ Uvicorn**

PyTests

**LangChain
e LangGraph**

Pydantic v2

Frontend

VS Code Extension

Axios

Padrões de Agentes de IA Utilizados

Reflection

Auto-crítica em loop. O agente revisa seu próprio output e corrige se necessário. Evita outputs incompletos sem supervisão humana.

Tool Use

LLM convoca ferramentas determinísticas externas.
Garante auditabilidade — o mapeamento risco→teste é código Python rastreável.

RAG

Retrieval-Augmented Generation: recupera exemplos similares da base vetorial antes de gerar. Calibra o LLM com contexto organizacional real.

Human-in-the-Loop

Pausa o pipeline para coletar input do dev. Mantém o humano no controle das decisões de contexto.

Avaliação da Solução

Avaliação Geral da Solução - Overview

- Mesmo requisito foi submetido a dois fluxos:
 - Agente multiagente proposto
 - LLM generalista sem orquestração
- Suíte com 10 casos de teste
- As respostas foram avaliadas com base em:
 - Rastreabilidade entre requisito, regras, riscos e cenários
 - Qualidade dos riscos identificados
 - Cobertura de cenários positivos, negativos e de borda
 - Aderência à estrutura Gherkin
 - Alinhamento entre tipos de teste e justificativas
 - Consistência de domínio ou ausência de escopo

Objetivo: comparação da arquitetura multiagente em casos de teste mais completos, estruturados e úteis para orientar desenvolvedores

Avaliação Geral da Solução - Metodologia

1. Geração dos outputs (Claude Opus 4.8)

- 1.1. Multi-Agente (TestGen)
- 1.2. LLM generalista pura

2. Coleta dos dados

- 2.1. Os outputs de ambos os testes são serializados arquivos de entradas para o Notebook em .ipynb
- 2.2. Definição dos critérios para a régua de avaliação

3. Avaliação no notebook (abordagem híbrida)

- 3.1. Embedding (BGE-M3)
- 3.2. LLM-as-judge (Claude Opus 4.8)

4. Escala de interpretação

- 4.1. Scores mapeados para rótulos padronizados:
 - 4.1.1. 0.0 não atende
 - 4.1.2. 0.2 muito pouco
 - 4.1.3. 0.4 pouco
 - 4.1.4. 0.6 parcialmente
 - 4.1.5. 0.8 bem
 - 4.1.6. 1.0 totalmente

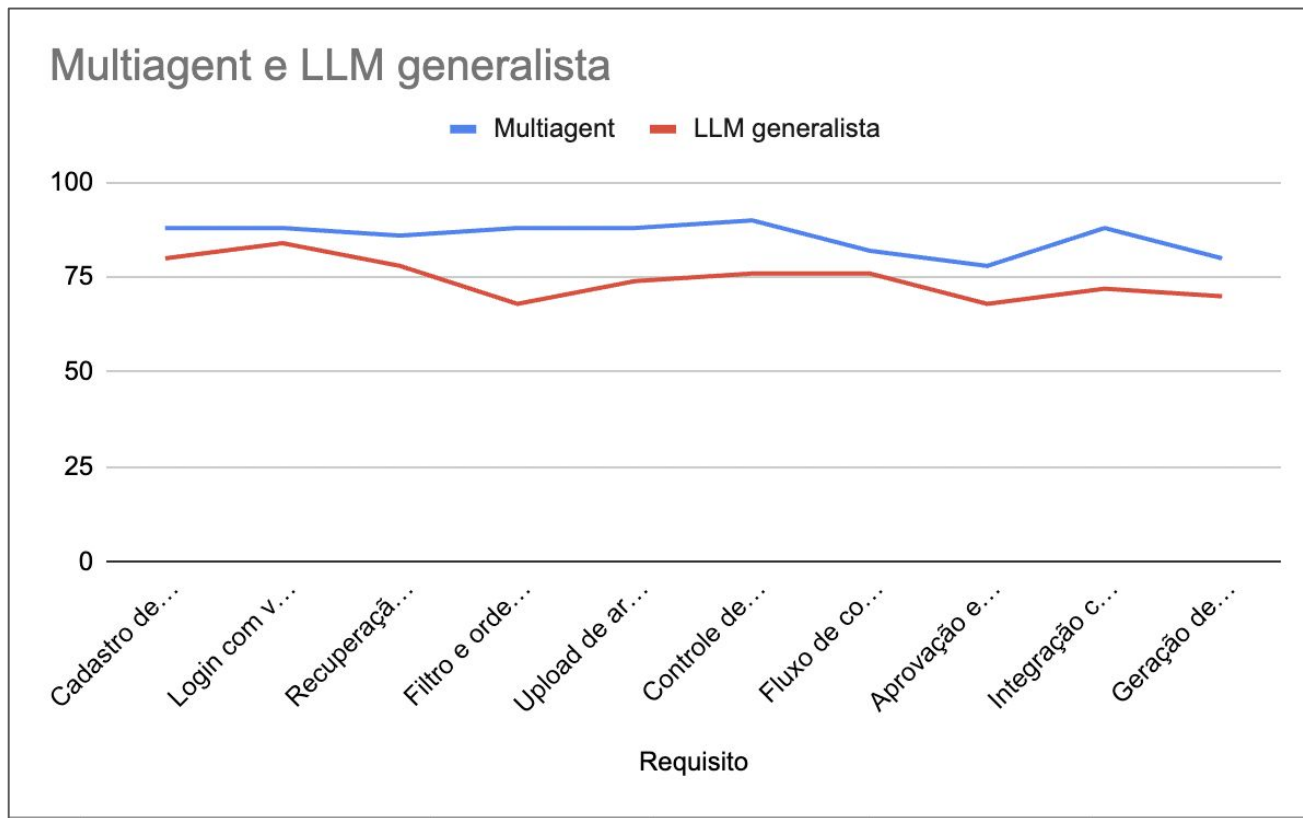
5. Resultado

- 5.1. Export de um arquivo xls com as métricas por requisitos
- 5.2. Análise manual dos scores gerados
- 5.3. Multiplicação dos scores para obter a % resultante

Avaliação Geral da Solução - Benchmark

#	Requisito	Complexidade	Multiagente	Generalista	Vantagem
1	Cadastro de usuário	Simple	88%	80%	8%
2	Login com validação	Simple	88%	84%	4%
3	Recuperação de senha	Simple	86%	78%	8%
4	Filtro e ordenação	Médio	88%	68%	20%
5	Upload de arquivo	Médio	88%	74%	14%
6	Controle de permissões	Médio	90%	76%	14%
7	Fluxo de compra	Médio	82%	76%	6%
8	Aprovação em múltiplas etapas	Complexo	78%	68%	10%
9	Integração com API externa	Complexo	88%	72%	16%
10	Geração de relatório	Complexo	80%	70%	10%

Avaliação Geral da Solução - Benchmark



Avaliação da Solução - Individual

- **Agent 1 Context Gatherer (15 amostras divididas em 3):**
 - **Precisão da Classificação (a: 5/5):** medir se o agente distingue corretamente o que é uma feature de software do que não é
 - **Precisão da Detecção de "Prontos" (b: 5/5):** medir se o agente decide corretamente quando tem contexto suficiente para prosseguir
 - **Recall Médio do Tópico (c: 3/5):** medir se as perguntas geradas cobrem os tópicos esperados

Avaliação da Solução - Individual

- **Agent 3 Risk Analyst (5 amostras):**
 - **Cobertura de Riscos Esperados ($\geq 90\%$):** mediu se o agente identifica corretamente todos os riscos relevantes da feature
 - **Qualidade dos Riscos Gerados ($\geq 90\%$):** mediu se o agente não inventa riscos que não existem
 - **Nível de Severidade Acertado ($\sim 80\%$):** mediu se o agente classifica corretamente a gravidade geral da feature (Baixa/Média/Alta/Crítica)
 - **Confusão de Contexto ($\sim 10\%$):** quanto dos erros são riscos que pertencem a outra feature

Avaliação da Solução - Individual

- **Agent 4 Test Strategist:**

Devido à natureza subjetiva da avaliação, optou-se por mensurá-la por meio da análise de terceiros. Foram realizados 10 testes na plataforma com features diversas; todas as respostas geradas foram coletadas e encaminhadas a 2 profissionais de QA, responsáveis por avaliar os cenários de teste produzidos. Além disso, as mesmas respostas foram submetidas a 2 LLMs, que também avaliaram os cenários, permitindo confrontar a percepção humana com a análise automatizada.

Avaliação de Terceiros

João Samuel

QA · 4 anos

A maioria dos casos faz sentido, porém ainda existem algumas alucinações. Boa parte desses casos de teste poderiam ser aproveitados e ajudariam bastante.

Carlos Henrique

QA · 5 anos

Faz sentido usar a ferramenta, mas com algumas ressalvas: vários critérios estão muito qualitativos, o que dificulta transformá-los em asserts automatizados. Também notei thresholds sem embasamento (como o "5 ou mais tentativas"), e nesses casos, em vez de assumir um número, eu amarraria a um requisito definido.

Claude Opus 4.8

3 meses de existência

Os casos de teste fazem sentido e a estrutura é profissional, cobrindo bem falso positivo/negativo, mas têm critérios vagos, thresholds inventados e redundância. Usaria como acelerador de uma primeira versão, nunca como entregável final sem revisão humana.

Gemini 3.1 Pro.

4 Meses de existência

O agente faz um ótimo trabalho ao mapear riscos reais e cobrir cenários complexos, entregando uma base muito sólida e economizando o tempo da equipe. A ressalva prática é apenas na hora da automação.

Avaliação da Solução - Individual

- **Agent 5 Documenter & Critic:**
- Foram criadas entradas simulando a saída dos agentes anteriores
- Cada entrada continha feature, riscos, criticidade, tipos de teste e cenários
- O objetivo foi verificar se o agente aprovava estratégias coerentes e rejeitava falhas críticas



Conclusão

Conclusão

- O conhecimento tácito do QA pode se tornar um pipeline padronizado e auditável, com resultado consistente entre projetos.
- A Matriz de Decisão (tool use) fornece rastreabilidade risco → teste → cenário; o LLM cuida da linguagem e dos cenários BDD.
- O RAG calibrou riscos e criticidade com decisões já validadas pela organização, mitigando a alucinação da LLM.
- Com essa estratégia, o planejamento de testes pode converter horas em minutos, direto no ambiente do dev (extensão VSCode) ou via API.

Referências

Referências

ADZIC, Gojko. Specification by Example: How Successful Teams Deliver the Right Software. Greenwich: Manning Publications, 2011.

ANTHROPIC. Building Effective Agents. Anthropic Engineering, 2024. Disponível em: <https://www.anthropic.com/engineering/building-effective-agents>.

BARBOZA, Eduardo Carlos. test-feature-classifier. GitHub, 2026. Disponível em: <https://github.com/Edu4rdoCarlos/test-feature-classifier>.

LEWIS, Patrick et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: Proceedings of the 34th International Conference on Neural Information Processing Systems (NeurIPS). Virtual Event: Curran Associates, 2020. v. 33, p. 9459-9474. Disponível em: <https://arxiv.org/abs/2005.11401>.

LIMA, Gabrielle; SILVA, Diógenes; MAR, Carlos; CORONEL, Diego. Automated Test Case Generation in a Real-World System Using a Customized AI Agent: An Experience Report. In: SBQS 2025, 2025, São José dos Campos. Proceedings [...]. 2025.

MCMINN, Phil. Search-based software testing: past, present and future. In: IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), 2011, Berlin. Proceedings [...]. Piscataway: IEEE, 2011. p. 153-163. Disponível em: <https://philmminn.com/publications/mcminn2011.pdf>.

SHINN, Noah et al. Reflexion: Language Agents with Verbal Reinforcement Learning. In: Proceedings of the 37th International Conference on Neural Information Processing Systems (NeurIPS). New Orleans: Curran Associates, 2023. Disponível em: <https://arxiv.org/abs/2303.11366>.

WANG, Jason; SULEIMAN, Basem; ALIBASA, Muhammad Johan. Automated Unit Test Case Generation: A Systematic Literature Review. arXiv:2504.20357v1 [cs.SE], 2025. Disponível em: <https://arxiv.org/pdf/2504.20357v1>.

YAO, Shunyu et al. ReAct: Synergizing Reasoning and Acting in Language Models. In: International Conference on Learning Representations (ICLR), 2023, Kigali. Proceedings [...]. 2023. Disponível em: <https://arxiv.org/abs/2210.03629>.